Programa:	Ingenieria de Software	**Asignatura:**	Programación Orientada a Objetos
Docente:	Maicol Ruidiaz	**Fecha:**	24/02/2026

Material de Apoyo
Programación orientada a Objetos - Java

Paradigmas y estilos de programación

- Imperativo: instrucciones paso a paso.
- Declarativo: describe el resultado esperado.
- POO: organiza en clases y objetos.

```
public class Calculadora {

    public int sumar(int a, int b) {
       return a+b;
    }
}
```

Conclusión: La POO facilita modularidad y reutilización.

Características de la POO

- Abstracción, Encapsulación, Herencia, Polimorfismo.

```
public class Animal {

   public void sonido() {
       System.out.println("Sonido genérico");
   }

}

class Perro extends Animal {

    @Override
   public void sonido() {
        System.out.println("Guau!");
   }

}
```

Conclusión: Estas características permiten flexibilidad y escalabilidad.

Principios de la POO

- Abstracción: ocultar detalles.
- Encapsulación: proteger datos.
- Herencia: reutilizar código.
- Polimorfismo: múltiples formas de un método. **Conclusión:** Son la base del diseño orientado a objetos.

Clases, objetos, atributos y métodos

```
public class Persona {
    String nombre;
    int edad;

   public void saludar() {
        System.out.println("Hola, soy " + nombre);
    }
}

Persona p = new Persona();
p.nombre = "Ana";
p.saludar();
```

Conclusión: Los objetos son instancias de clases.

POO en Java: compilación y ejecución

- javac MiClase.java → compila.
- java MiClase → ejecuta.
- **Conclusión:** Java transforma código fuente en bytecode para la JVM.

Tipos de datos y modificadores de acceso

- Primitivos: int, double, char, boolean.
- Acceso: public, private, protected.

```
public class Cuenta {

    private double saldo;

    public void depositar(double monto) {
        this.saldo += monto;
    }
}
```

Conclusión: Los modificadores controlan la visibilidad.

Lectura y escritura de datos

```
Scanner sc = new Scanner(System.in);
System.out.print("Ingrese nombre: ");
String nombre = sc.nextLine();
```

Conclusión: Java facilita interacción con el usuario.

Encapsulación, Constructores, Setters y Getters

```
public class Estudiante {
    private String nombre;

    Estudiante(String n) {
        this.nombre = n;
   }

   public String getNombre() {
       return nombre;
    }

    public void setNombre(String n) {
      this.nombre = n;
     }
}
```

Conclusión: Encapsulación protege datos y asegura consistencia.

Ejercicio

- Crear clase Libro con atributos título y autor.
- Implementar métodos para mostrar información y cambiar autor.
- **Conclusión:** La práctica refuerza el aprendizaje de POO.